

Open Source Beginner Guide: Git & GitHub Commands (Codess Cohort 8)

A start-to-finish guide to every Git/GitHub command you need to contribute to open source, what each one means, and a walkthrough of your first real PR to [firstcontributions/first-contributions](#) titled "**Codess cohort 8**".

1. The mental model (read this first)

Term	What it means
Git	A tool on <i>your computer</i> that records every change to your files (what, when, who).
GitHub	A website that hosts Git repositories online so people can collaborate.
Repository (repo)	A project folder tracked by Git, including its full history.
Working directory	The files you see and edit on your machine.
Staging area	A "shopping cart" of changes you've chosen to include in the next commit.
Commit	A saved snapshot of staged changes, with a message explaining it.
Branch	A separate line of work. You make changes on a branch so <code>main</code> stays clean.
Remote	A copy of the repo hosted somewhere else (e.g. GitHub).
<code>origin</code>	The default name for <i>your</i> remote (usually your fork).
<code>upstream</code>	The conventional name for the <i>original</i> project you forked from.
Fork	Your own copy of someone else's repo, under your GitHub account. You can push to it.
Clone	Downloading a repo from GitHub to your computer.
Pull Request (PR)	A request asking the maintainers to merge changes from your branch into their project.
Issue	A note on a project describing a bug, feature request, or question. Not code.
Merge conflict	When two changes edit the same lines and Git needs you to choose which to keep.

The flow of a change:

```
edit files
  | git add
  ▼
[staging area]
  | git commit
  ▼
[local repo]
  | git push
  ▼
[your fork on GitHub]
  | open Pull Request
  ▼
[original repo]
```

Open source workflow in one picture:

```
1. original repo (upstream) — click Fork → your fork (origin)
2. your fork (origin)       — git clone → your laptop
3. your laptop              — git push  → your fork (origin)
4. your fork (origin)       — Pull Request → original repo (upstream)
```

2. One-time setup

```
git --version
```

Checks Git is installed. If not, download it from <https://git-scm.com/downloads>.

```
git config --global user.name "Nandika Gupta"
git config --global user.email "you@example.com"
```

Tells Git who you are. This name/email is stamped on every commit. Use the email linked to your GitHub account so commits show up on your profile.

```
git config --global init.defaultBranch main
```

Makes new repos start on a branch called `main` (the modern default).

```
git config --list
```

Shows all your current Git settings.

Authentication: GitHub no longer accepts your password on the command line. Either:

- Install the [GitHub CLI](#) and run `gh auth login` (easiest), or
- Create a Personal Access Token (GitHub → Settings → Developer settings → Tokens) and paste it when asked for a password, or
- Set up an SSH key (`ssh-keygen -t ed25519 -C "you@example.com"` , then add the `.pub` key under GitHub → Settings → SSH keys).

3. Terminal basics you'll use alongside Git

Command	Meaning
<code>pwd</code>	Print where you are (current folder).
<code>ls</code> (<code>dir</code> on Windows CMD)	List files in the folder.
<code>cd folder-name</code>	Move into a folder. <code>cd ..</code> goes up one level.
<code>mkdir name</code>	Make a new folder.
<code>touch file.txt</code>	Create an empty file (Mac/Linux/Git Bash).
<code>code .</code>	Open the current folder in VS Code (if installed).

4. Getting a project onto your machine

```
git clone https://github.com/<your-username>/<repo>.git
```

Downloads a full copy of the repo (all files + history) into a new folder. Clone **your fork**, not the original, so you can push to it.

```
cd <repo>
```

Enter the project folder; Git commands only work inside a repo.

```
git init
```

Turns the *current* folder into a brand-new Git repo (creates a hidden `.git` folder). You only need this when starting your own project from scratch, not when cloning.

```
git remote -v
```

Lists remotes and their URLs. After cloning your fork you'll see `origin`.

```
git remote add upstream https://github.com/<original-owner>/<repo>.git
```

Adds the original project as a second remote named `upstream`, so you can pull in its latest changes.

5. Checking what's going on

```
git status
```

Your most-used command. Shows your branch, which files are modified, staged, or untracked. Run it before and after everything.

```
git diff
```

Shows exactly which lines changed that are *not yet staged*. `git diff --staged` shows what *is* staged.

```
git log
git log --oneline
git log --oneline --graph --all
```

Shows commit history. `--oneline` = one line per commit; `--graph --all` draws branches.

6. Branches

```
git branch
```

Lists local branches; `*` marks the one you're on. `git branch -a` includes remote branches.

```
git switch -c my-branch-name
# older equivalent: git checkout -b my-branch-name
```

Creates a new branch and moves onto it. Always make a new branch for each contribution.

```
git switch main
# older equivalent: git checkout main
```

Moves to an existing branch.

```
git branch -d my-branch-name
```

Deletes a local branch (after it's merged). `-D` forces deletion.

7. Saving your work

```
git add Contributors.md    # stage one file
git add .                  # stage everything in this folder and below
git add -A                 # stage every change in the whole repo, including deletions
```

Moves changes into the staging area. In open source, prefer adding **specific files** so you don't accidentally commit junk.

```
git restore --staged file.txt
# older equivalent: git reset file.txt
```

Un-stages a file (keeps your edits).

```
git restore file.txt
```

Throws away your un-staged edits to a file (careful: not recoverable).

```
git commit -m "Add Nandika Gupta to Contributors list"
```

Saves the staged changes as a snapshot. Write messages in the imperative: "Add...", "Fix...", "Update..."

```
git commit --amend
```

Edits the most recent commit (message or contents). Only do this **before** pushing, or on your own branch.

```
git reset HEAD~1
```

Undoes the last commit but keeps the changes in your files.

```
git rm file.txt            # delete file and stage the deletion
git rm --cached file.txt   # stop tracking but keep the file on disk
git mv old.txt new.txt     # rename/move and stage it
```

8. Syncing with GitHub

```
git push -u origin my-branch-name
```

Uploads your branch's commits to your fork. `-u` remembers the link, so next time just `git push`.

```
git pull
```

Downloads new commits from the remote branch and merges them into your current branch (= `git fetch` + `git merge`).

```
git fetch upstream
```

Downloads the latest from the original project **without** changing your files.

```
git switch main
git pull upstream main
git push origin main
```

The standard "keep my fork up to date" routine. Do this before starting new work.

9. Combining work & fixing conflicts

```
git merge upstream/main
```

Brings changes from another branch into your current branch.

If you see `CONFLICT`, open the file and look for:

```
<<<<<<< HEAD
your version
=====
their version
>>>>>>> upstream/main
```

Edit it to the final text you want, delete the markers, then:

```
git add <file>
git commit
```

```
git rebase upstream/main
```

Replays your commits on top of the latest upstream (cleaner history). Only rebase branches that are yours. If you rebased an already-pushed branch, you'll need `git push --force-with-lease`.

```
git merge --abort    # or: git rebase --abort
```

Bail out of a merge/rebase that went wrong and go back to where you were.

10. Handy extras

```
git stash          # temporarily shelve uncommitted changes
git stash pop      # bring them back
git revert <commit-sha> # new commit that undoes an old one (safe)
git show <commit-sha>  # see what a specific commit changed
git blame file.txt     # see who last changed each line
```

`.gitignore`: a file listing things Git should never track (e.g. `node_modules/`, `.env`).

11. Your first contribution: first-contributions → PR "Codess cohort 8"

The [first-contributions](#) repo exists so beginners can practise the whole flow safely. The task is to **add your name to** `Contributors.md` and open a PR.

Step 1: Fork

Go to <https://github.com/firstcontributions/first-contributions> and click **Fork** (top right) → **Create fork**. You now have <https://github.com/Nandika-Gupta/first-contributions>.

Step 2: Clone *your fork*

```
git clone https://github.com/Nandika-Gupta/first-contributions.git
cd first-contributions
```

Step 3: Create a branch

```
git switch -c codess-cohort-8
```

Step 4: Make the change

Open `Contributors.md` in any editor. Add one line **somewhere in the middle** (not the very top or bottom, which avoids conflicts with others), matching the existing format:

```
- [Nandika Gupta](https://github.com/Nandika-Gupta) - My first open-source contribution with
Codess Cohort 8!
```

Check your change:

```
git status
git diff
```

Step 5: Stage and commit

```
git add Contributors.md
git commit -m "Add Nandika Gupta to Contributors list"
```

Step 6: Push to your fork

```
git push -u origin codess-cohort-8
```

Step 7: Open the Pull Request

1. Go to your fork on GitHub. A yellow banner shows **Compare & pull request**; click it.
2. Check that the base repository is `firstcontributions/first-contributions` with base `main`, and the head repository is `Nandika-Gupta/first-contributions` with compare `codess-cohort-8`.
3. **Title:** `Codess cohort 8`
4. **Description** (example): `Adding my name to Contributors.md as my first open-source contribution, as part of Codess Cohort 8.`
5. Click **Create pull request**. 🎉

Step 8: After you open it

- A bot/maintainer usually merges it automatically within a short time; you'll get an email.
- If asked to change something: edit, `git add`, `git commit`, `git push`. The PR updates by itself.
- If GitHub says there's a conflict, sync with upstream (section 8) and merge (section 9).

Step 9: Clean up (optional)

```
git switch main
git remote add upstream https://github.com/firstcontributions/first-contributions.git
git pull upstream main
git branch -d codess-cohort-8
```

12. Etiquette & next steps

- **Read** `README`, `CONTRIBUTING.md`, and the **Code of Conduct** of every project first.
- **Look for labels** like `good first issue`, `help wanted`, `documentation`. Also try goodfirstissue.dev and up-for-grabs.net.
- **Comment on an issue before working on it** ("I'd like to work on this, my plan is..."). Wait to be assigned if the project assigns issues. Don't claim many issues at once.
- **Check a project is active:** recent commits, maintainers replying to issues/PRs.
- **Keep PRs small and focused:** one fix per PR. Link the issue with `Fixes #123` in the description.
- **Use a draft PR** if you want early feedback on unfinished work.

- **Be patient:** maintainers are often volunteers. A polite follow-up after ~1 week is fine.
- **If you use AI,** understand and test every line you submit; you're responsible for it.
- Contribution isn't only code: docs, tests, bug reports, reviewing, and helping others all count.

13. Cheat sheet

Goal	Command
Copy repo to laptop	<code>git clone <url></code>
See what changed	<code>git status</code> , <code>git diff</code>
New branch	<code>git switch -c <name></code>
Switch branch	<code>git switch <name></code>
Stage	<code>git add <file></code>
Commit	<code>git commit -m "message"</code>
Upload	<code>git push -u origin <branch></code>
Get latest	<code>git pull</code> / <code>git fetch upstream</code>
Add original repo	<code>git remote add upstream <url></code>
Combine	<code>git merge <branch></code>
Shelve work	<code>git stash</code> / <code>git stash pop</code>
Undo last commit (keep edits)	<code>git reset HEAD~1</code>
Undo a pushed commit safely	<code>git revert <sha></code>
History	<code>git log --oneline</code>